**INSTITUTO FEDERAL DE SÃO PAULO**
Câmpus Hortolândia

LABORATÓRIO

# Computador Cesar

Programação em linguagem de montagem com o montador Daedalus

AUTOR
**Gabriel Cabral Wallendzu**
HT3080251

DISCIPLINA
Sistemas de Computação 1 de Automação

PROFESSOR
Carlos Eduardo Pagani

Hortolândia – SP | 2026

# Sumário

Resumo ..... 3
1 Introdução ..... 3
2 Objetivos ..... 4
3 Metodologia ..... 4
4 Desenvolvimento ..... 4
4.1 Soma de n posições: do Ramses ao Cesar ..... 4
4.2 Exercícios resolvidos do laboratório ..... 8
4.3 Exercícios propostos ..... 11
4.4 Trabalho avaliativo ..... 15
5 Testes realizados ..... 16
6 Divisão das tarefas ..... 17
7 Conclusão ..... 17
8 Repositório e referências ..... 18

## Resumo

Este relatório reúne a resolução do laboratório “Computador Cesar”, realizado com o montador Daedalus e o simulador Cesar. O trabalho começa comparando o Cesar com o Ramses por meio de um programa que soma n posições de memória, reescrito em cinco versões que exploram instruções e modos de endereçamento exclusivos do Cesar. Em seguida são estudados os exemplos resolvidos do laboratório (soma de 32 bits, movimentação de bloco, pesquisa em vetor e leitura de teclado com eco no visor), resolvidos os três exercícios propostos (busca em vetor, inicialização de vetor e manipulação de bits) e desenvolvido o trabalho avaliativo, que exibe nome e RA no visor e apresenta um prompt para resposta S/N.

## 1 Introdução

O Cesar é um computador hipotético inspirado no PDP-11 e representa um salto em relação às máquinas estudadas antes (Neander, Ahmes e Ramses). Ele trabalha com dados e endereços de 16 bits (64 KB de memória), representa números em complemento de dois, organiza a memória em bytes no formato big endian e oferece oito registradores de 16 bits: R0 a R5 de uso geral, R6 como ponteiro de pilha (SP) e R7 como contador de programa (PC). Seu registrador de estado tem quatro flags: negativo (N), zero (Z), carry (C) e overflow (V). Há ainda dois periféricos mapeados em memória: um visor de 36 caracteres e um teclado.

A tabela abaixo resume as diferenças entre Ramses e Cesar que mais aparecem neste trabalho.

| Característica | Ramses | Cesar |
|---|---|---|
| Largura de dados e endereços | 8 bits (256 posições) | 16 bits (64 KB), big endian |
| Registradores | RA, RB e RX | R0 a R5 (uso geral), R6 = SP, R7 = PC |
| Flags | N, Z e C | N, Z, C e V |
| Formato das instruções | Carga e armazenamento (LDR/STR) e operações com registrador e operando | MOV, ADD, SUB etc. com dois operandos, que podem estar em memória |
| Modos de endereçamento | Direto, indireto, imediato e indexado | Oito modos (registrador, pós-incrementado, pré-decrementado, indexado e versões indiretas) mais imediato, absoluto e relativo via R7 |
| Desvios | Endereço absoluto (JMP, JZ, JC) | Desvios condicionais relativos (BEQ, BNE, BGT...), BR, JMP e SOB |
| Sub-rotinas | JSR, com retorno guardado no 1º byte da rotina | JSR/RTS com qualquer registrador de ligação e uso da pilha |
| Tamanho das instruções | 2 bytes | De 1 a 6 bytes, conforme os operandos |
| Passo dos ponteiros | 1 byte por posição | 2 bytes por palavra nos modos com incremento/decremento |

| Característica | Ramses | Cesar |
|---|---|---|
| Entrada e saída | Não possui | Visor e teclado mapeados em memória (65498 a 65535) |

# 2 Objetivos

O objetivo geral é resolver o laboratório “Computador Cesar” usando o montador Daedalus no modo Cesar e o simulador correspondente. Como objetivos específicos, busca-se:

- comparar a linguagem do Ramses com a do Cesar, observando como instruções e endereçamentos alteram o tamanho e a posição do código na memória;
- praticar os modos de endereçamento do Cesar, os desvios relativos, a instrução SOB e as sub-rotinas com JSR e RTS;
- trabalhar com números de 32 bits, vetores, bits individuais e caracteres ASCII;
- usar o visor e o teclado, que são periféricos mapeados em memória.

# 3 Metodologia

Cada programa foi escrito no Daedalus com a máquina alvo Cesar, montado e executado no simulador, conferindo ao final os registradores e as posições de memória envolvidas. Foi adotada a seguinte convenção: o programa começa na posição 0 (org 0) e os dados ficam a partir da posição 1024 (org 1024), pois a área de programa vai de 0 a 1023. Como o Cesar é big endian, uma palavra declarada com dw ocupa dois bytes consecutivos, com o byte mais significativo no endereço menor, e por isso os endereços de palavras avançam de 2 em 2.

Os desvios foram escritos com rótulos, e o montador calcula o deslocamento relativo. Quando o deslocamento numérico é relevante para a explicação, ele é citado no texto. Todos os programas possuem comentários após o ponto e vírgula, e os códigos-fonte completos estão no repositório do GitHub indicado ao final do relatório.

# 4 Desenvolvimento

Os códigos abaixo também estão disponíveis, em arquivos separados, no repositório do GitHub: **https://github.com/GCWKGSWTGW/Daedalus-Ahmes-Ramses**

## 4.1 Soma de n posições: do Ramses ao Cesar

> *Criar um programa que some n posições consecutivas de memória a partir de um endereço e e armazene o resultado em uma posição de memória.*

O enunciado fornece a solução em Ramses, que serve de ponto de partida. Ela usa três registradores (RA para o total, RX como ponteiro e RB como contador), o endereçamento indexado (ADD A 0,X) e

um laço que testa o contador com JZ. Os dados são n (posição 128), e (posição 129) e o total (posição 130).

```
; Base do enunciado em Ramses: soma de n posicoes consecutivas
org 0
    LDR A #0          ; total = 0
    LDR X 129         ; X = endereco inicial (conteudo da posicao 129)
    LDR B 128         ; B = n (conteudo da posicao 128)
LACO:
    JZ FIM            ; contador = 0 ? termina
    ADD A 0,X         ; soma o elemento apontado por X
    ADD X #1          ; avanca o ponteiro (1 byte)
    SUB B #1          ; contador - 1
    JMP LACO
FIM:
    STR A 130         ; grava o total
    HLT
```

No Cesar, o mesmo algoritmo foi escrito em uma versão inicial e refinado em quatro passos. Em todas as versões os dados são os mesmos: n na posição 1024, o endereço inicial e na posição 1026 e o total na posição 1028, com o vetor de teste a partir da posição 1030.

```
; Area de dados (igual nas cinco versoes)
org 1024
TAM:    dw 5          ; n = numero de posicoes
ENDINI: dw 1030       ; endereco inicial do vetor
TOTAL:  dw 0          ; resultado da soma

org 1030              ; vetor a ser somado
    dw 10
    dw 20
    dw 30
    dw 40
    dw 50
```

### Versão 0: tradução direta do Ramses

```
; Versao 0 - traducao direta do Ramses
org 0
    MOV #0, R0        ; total = 0
    MOV 1026, R2      ; ponteiro = endereco inicial
    MOV 1024, R1      ; contador = n
LACO:
    BEQ FIM           ; contador = 0 ? termina
    ADD (R2), R0      ; soma o elemento apontado por R2
    ADD #2, R2        ; ponteiro + 2 (palavra = 2 bytes)
    SUB #1, R1        ; contador - 1
    JMP LACO
FIM:
    MOV R0, 1028      ; grava o total
    HLT
```

A versão 0 mantém a estrutura do Ramses: o total é zerado com MOV #0, o contador é decrementado com SUB #1 e o laço retorna com JMP. As diferenças já aparecem na carga dos valores, que agora é feita com MOV entre memória e registrador (MOV 1026, R2), no ponteiro, que avança de 2 em 2 porque cada palavra tem 2 bytes (ADD #2, R2), e no acesso ao elemento, feito pelo modo registrador indireto (ADD (R2), R0), que substitui o endereçamento indexado do Ramses.

### Versão 1: CLR, DEC e BR

```
; Versao 1 - CLR, DEC e BR
org 0
    CLR R0            ; total = 0 (instrucao menor que MOV #0)
    MOV 1026, R2
```

```
    MOV 1024, R1
LACO:
    BEQ FIM
    ADD (R2), R0
    ADD #2, R2
    DEC R1            ; contador - 1 (substitui SUB #1)
    BR LACO           ; desvio relativo (substitui JMP)
FIM:
    MOV R0, 1028
    HLT
```

Foram trocadas três instruções por equivalentes mais curtas: CLR R0 no lugar de MOV #0, R0, DEC R1 no lugar de SUB #1, R1 e BR (desvio relativo) no lugar de JMP, que carrega um endereço absoluto de 16 bits. Cada troca elimina uma palavra de operando e, com isso, o programa encolhe e as instruções seguintes mudam de endereço.

### Versão 2: BNE no final do laço

```
; Versao 2 - BNE no final do laco
org 0
    CLR R0
    MOV 1026, R2
    MOV 1024, R1
    BEQ FIM           ; testa n = 0 uma unica vez, antes do laco
LACO:
    ADD (R2), R0
    ADD #2, R2
    DEC R1
    BNE LACO          ; repete enquanto o contador nao zerar
FIM:
    MOV R0, 1028
    HLT
```

O teste de n = 0 passa a ser feito uma única vez, antes do laço, e o fim do laço usa BNE, que repete enquanto o contador não zerar. Assim o corpo do laço deixa de ter o BEQ inicial e o BR final: são quatro instruções por volta, e não mais cinco.

### Versão 3: endereçamento pós-incrementado

```
; Versao 3 - enderecamento pos-incrementado
org 0
    CLR R0
    MOV 1026, R2
    MOV 1024, R1
LACO:
    BEQ FIM
    ADD (R2)+, R0     ; soma e ja incrementa R2 em 2
    DEC R1
    BR LACO
FIM:
    MOV R0, 1028
    HLT
```

O modo (R2)+ lê o elemento apontado por R2 e, depois do uso, incrementa R2 em 2 automaticamente. A instrução ADD #2, R2 deixa de ser necessária, e o laço fica com quatro instruções e quatro bytes a menos de código.

### Versão 4: instrução SOB

```
; Versao 4 - instrucao SOB
org 0
    CLR R0
    MOV 1026, R2
    MOV 1024, R1
```

```
    BEQ FIM
LACO:
    ADD (R2)+, R0
    SOB R1, LACO      ; R1 - 1; se diferente de zero, volta ao laco
FIM:
    MOV R0, 1028
    HLT
```

A instrução SOB R1, LACO decrementa o contador e desvia de volta ao laço se ele não zerou, reunindo DEC e BNE em uma só instrução. O laço passa a ter apenas duas instruções, e o teste de n = 0 antes dele continua necessário, pois com contador inicial zero o SOB decrementaria até 65535.

### Posição das instruções na memória

| Endereço | Versão 0 | Versão 1 | Versão 2 | Versão 3 | Versão 4 |
|---|---|---|---|---|---|
| 0 | MOV #0,R0 | CLR R0 | CLR R0 | CLR R0 | CLR R0 |
| 2 | | MOV 1026,R2 | MOV 1026,R2 | MOV 1026,R2 | MOV 1026,R2 |
| 4 | MOV 1026,R2 | | | | |
| 6 | | MOV 1024,R1 | MOV 1024,R1 | MOV 1024,R1 | MOV 1024,R1 |
| 8 | MOV 1024,R1 | | | | |
| 10 | | BEQ +10 | BEQ +10 | BEQ +6 | BEQ +4 |
| 12 | BEQ +14 | ADD (R2),R0 | ADD (R2),R0 | ADD (R2)+,R0 | ADD (R2)+,R0 |
| 14 | ADD (R2),R0 | ADD #2,R2 | ADD #2,R2 | DEC R1 | SOB R1,4 |
| 16 | ADD #2,R2 | | | BR -8 | MOV R0,1028 |
| 18 | | DEC R1 | DEC R1 | MOV R0,1028 | |
| 20 | SUB #1,R1 | BR -12 | BNE -10 | | HLT |
| 22 | | MOV R0,1028 | MOV R0,1028 | HLT | |
| 24 | JMP 12 | | | | |
| 26 | | HLT | HLT | | |
| 28 | MOV R0,1028 | | | | |
| 32 | HLT | | | | |

Os endereços em branco são palavras de operando da instrução anterior (por exemplo, o 4 da versão 1 é a palavra com o endereço 1026 da instrução que começa em 2). O deslocamento de um desvio é contado a partir do endereço da instrução seguinte: na versão 3, o BR está no endereço 16, a próxima instrução está em 18 e o laço volta ao BEQ do endereço 10, o que dá um deslocamento de −8.

### Efeito das alterações

| Versão | Tamanho do programa | Instruções por volta | Principal mudança |
|---|---|---|---|
| 0 | 33 bytes | 5 | Tradução direta, com JMP absoluto |

| Versão | Tamanho do programa | Instruções por volta | Principal mudança |
|---|---|---|---|
| 1 | 27 bytes | 5 | CLR, DEC e BR deixam as instruções mais curtas |
| 2 | 27 bytes | 4 | Teste de zero fora do laço e BNE no final |
| 3 | 23 bytes | 4 | Ponteiro incrementado junto com o acesso |
| 4 | 21 bytes | 2 | SOB faz a contagem e o desvio |

Em todas as versões o resultado é o mesmo: com n = 5 e o vetor 10, 20, 30, 40 e 50, a posição 1028 recebe 150 e o registrador R0 termina com 150. A cada passo, o programa fica menor, o que muda os endereços das instruções, e o laço executa menos instruções por volta, reduzindo também os acessos à memória.

**Resultado obtido:** posição 1028 = 150 (0096h), R0 = 150 e R2 = 1040 ao final, em todas as versões.

## 4.2 Exercícios resolvidos do laboratório

### Soma de variáveis de 32 bits

> *Somar duas variáveis de 32 bits, cada uma armazenada em duas palavras de 16 bits.*

Como os registradores têm 16 bits, cada número de 32 bits é dividido em parte alta e parte baixa. Soma-se primeiro a parte baixa; se o resultado ultrapassar 65535, a flag C é ativada. Como o MOV não altera o carry, ele sobrevive ao MOV R0, 1034 e ao MOV #0, R0, e a instrução ADC R0 soma esse carry ao registrador, que passa a valer 1 ou 0. Em seguida são somadas as partes altas, com o carry já incluído. O teste usa 125536 (parte alta 1, parte baixa 60000) mais 141072 (parte alta 2, parte baixa 10000), valores escolhidos para gerar carry na parte baixa.

```
; Soma de duas variaveis de 32 bits (cada uma em duas palavras)
org 1024
HI1: dw 1             ; var1: parte alta
LO1: dw 60000         ; var1: parte baixa  (var1 = 1 x 65536 + 60000)
HI2: dw 2             ; var2: parte alta
LO2: dw 10000         ; var2: parte baixa  (var2 = 2 x 65536 + 10000)
RHI: dw 0             ; resultado: parte alta
RLO: dw 0             ; resultado: parte baixa

org 0
    MOV 1026, R0      ; R0 = parte baixa de var1
    ADD 1030, R0      ; soma a parte baixa de var2 (C = 1 se estourar 16 bits)
    MOV R0, 1034      ; salva a parte baixa do resultado
    MOV #0, R0        ; zera R0 (MOV nao altera o carry)
    ADC R0            ; R0 = R0 + carry (vai um da soma anterior)
    ADD 1024, R0      ; soma a parte alta de var1
    ADD 1028, R0      ; soma a parte alta de var2
    MOV R0, 1032      ; salva a parte alta do resultado
    HLT
```

**Resultado obtido:** parte baixa em 1034 = 4464 (60000 + 10000 − 65536) e parte alta em 1032 = 4 (1 + 2 + 1 de carry).

### Mover n posições de memória em bloco

> *Copiar um bloco de n palavras de uma posição de origem para uma posição de destino, sem zerar a origem.*

Quando os blocos se sobrepõem, o sentido da cópia importa. Se o destino estiver acima da origem e a cópia for feita do início para o fim, as primeiras palavras copiadas sobrescrevem elementos da origem que ainda não foram lidos. Por isso o programa compara destino e origem (CMP R2, R1) e, se o destino for maior (BGT), copia de trás para frente: dobra R0 com ASL para obter o tamanho em bytes, soma esse valor aos dois ponteiros para que apontem para o fim dos blocos e usa o modo pré-decrementado (-(R1), -(R2)). Caso contrário, a cópia é feita no sentido crescente com o modo pós-incrementado. O teste usa origem em 1030 e destino em 1034, com sobreposição, para exercitar o caminho inverso.

```
; Mover n palavras de um bloco (sem zerar a origem)
org 1024
    dw 4              ; n = numero de palavras
    dw 1030           ; endereco inicial da origem
    dw 1034           ; endereco inicial do destino (sobrepoe a origem)

org 1030
    dw 11
    dw 22
    dw 33
    dw 44

org 0
    MOV 1024, R0      ; R0 = n
    MOV 1026, R1      ; R1 = origem
    MOV 1028, R2      ; R2 = destino
    CMP R2, R1        ; destino - origem
    BGT INVERSO       ; destino acima da origem: copia de tras para frente
FRENTE:
    MOV (R1)+, (R2)+  ; copia uma palavra no sentido crescente
    SOB R0, FRENTE
    HLT
INVERSO:
    ASL R0            ; n x 2 = tamanho do bloco em bytes
    ADD R0, R1        ; R1 aponta para o fim da origem
    ADD R0, R2        ; R2 aponta para o fim do destino
    MOV 1024, R0      ; restaura o contador em palavras
LACO_INV:
    MOV -(R1), -(R2)  ; decrementa os dois ponteiros e copia
    SOB R0, LACO_INV
    HLT
```

**Resultado obtido:** as posições 1034, 1036, 1038 e 1040 passam a conter 11, 22, 33 e 44, mesmo com a sobreposição; as posições 1030 e 1032 continuam com 11 e 22.

### Pesquisa em vetores

> *Determinar o maior valor de um vetor e sua posição relativa (1º, 2º, ..., n-ésimo).*

O primeiro elemento é assumido como o maior (MOV (R1)+, R2), e R3 guarda o valor do contador no momento em que o maior foi encontrado. O laço compara cada elemento com o maior atual: se for menor ou igual (BLE), pula a atualização; se for maior, atualiza R2 e R3. Ao final, a posição é calculada por n − R3 + 1. Em relação ao exemplo do laboratório, foi incluído um desvio logo após o DEC R0, para que um vetor de apenas um elemento não faça o SOB decrementar o contador a partir de zero.

```
; Maior valor de um vetor e sua posicao relativa
org 1024
    dw 8              ; n = tamanho do vetor
```

```
    dw 1032           ; endereco inicial do vetor
    dw 0              ; resultado: maior valor      (1028)
    dw 0              ; resultado: posicao relativa (1030)

org 1032
    dw 12
    dw 17
    dw 23
    dw 15
    dw 8
    dw 27
    dw 26
    dw 37

org 0
    MOV 1024, R0      ; R0 = tamanho
    MOV 1026, R1      ; R1 = ponteiro do vetor
    MOV (R1)+, R2     ; 1o elemento = maior provisorio
    MOV R0, R3        ; R3 = contador associado ao maior
    DEC R0            ; restam n - 1 elementos
    BEQ RESULTADO     ; vetor de 1 elemento
LACO:
    CMP (R1), R2      ; elemento - maior
    BLE PROXIMO       ; menor ou igual: mantem o maior
    MOV (R1), R2      ; novo maior
    MOV R0, R3        ; guarda o contador atual
PROXIMO:
    ADD #2, R1        ; avanca o ponteiro
    SOB R0, LACO
RESULTADO:
    MOV R2, 1028      ; maior valor
    MOV 1024, R4
    SUB R3, R4
    INC R4            ; posicao = n - contador + 1
    MOV R4, 1030      ; posicao relativa
    HLT
```

**Resultado obtido:** posição 1028 = 37 (maior valor) e posição 1030 = 8 (último elemento do vetor).

### Leitura do teclado e eco no visor

> *Ler caracteres do teclado e exibi-los em posições consecutivas do visor.*

O teclado ocupa dois endereços: 65498 guarda o estado (o valor 128 indica que há uma tecla) e 65499 guarda o código ASCII da tecla. O programa libera o teclado zerando o estado (CLR), espera até que o estado deixe de ser zero (TST e BEQ), copia o caractere para a posição atual do visor, que começa em 65500, e avança o ponteiro. Como o visor termina em 65535, o INC R1 que ultrapassa esse endereço faz R1 virar zero e ativa a flag Z; o BEQ seguinte volta ao ponto em que R1 recebe 65500, reiniciando a escrita no início do visor. Os deslocamentos gerados são −4 no teste do teclado, −18 no retorno ao início do visor e −16 no retorno à liberação do teclado.

```
; Leitura do teclado com eco no visor (e retorno ao inicio ao encher)
org 0
    MOV #65498, R3    ; R3 = endereco do estado do teclado
REINICIA:
    MOV #65500, R1    ; R1 = primeira posicao do visor
LIBERA:
    CLR (R3)          ; libera o teclado para uma nova tecla
ESPERA:
    TST (R3)
    BEQ ESPERA        ; estado 0: ainda nao ha tecla
    MOV 65499, (R1)   ; copia o caractere digitado para o visor
    INC R1            ; proxima posicao do visor
    BEQ REINICIA      ; R1 passou de 65535 e virou 0: visor cheio
    BR LIBERA
```

**Resultado obtido:** cada tecla digitada aparece no visor, na posição seguinte à anterior, e a escrita volta ao início quando as 36 posições se enchem.

## 4.3 Exercícios propostos

### Exercício 1: posição de um valor em um vetor

> *Determinar a posição de um determinado valor (em ASCII) dentro de um vetor.*

O vetor contém os códigos ASCII das letras A, B, C e D, e o valor procurado é 67 (a letra C). O laço compara cada elemento com o valor procurado usando CMP (R1)+, R2, que já avança o ponteiro, e mantém em R3 a posição atual, que começa em 1. Se houver igualdade, o programa desvia para ACHOU e grava R3 na posição 1030; se o contador chegar a zero sem igualdade, R3 é zerado, de modo que o resultado 0 indica que o valor não está no vetor. O teste inicial de n = 0 evita percorrer um vetor vazio.

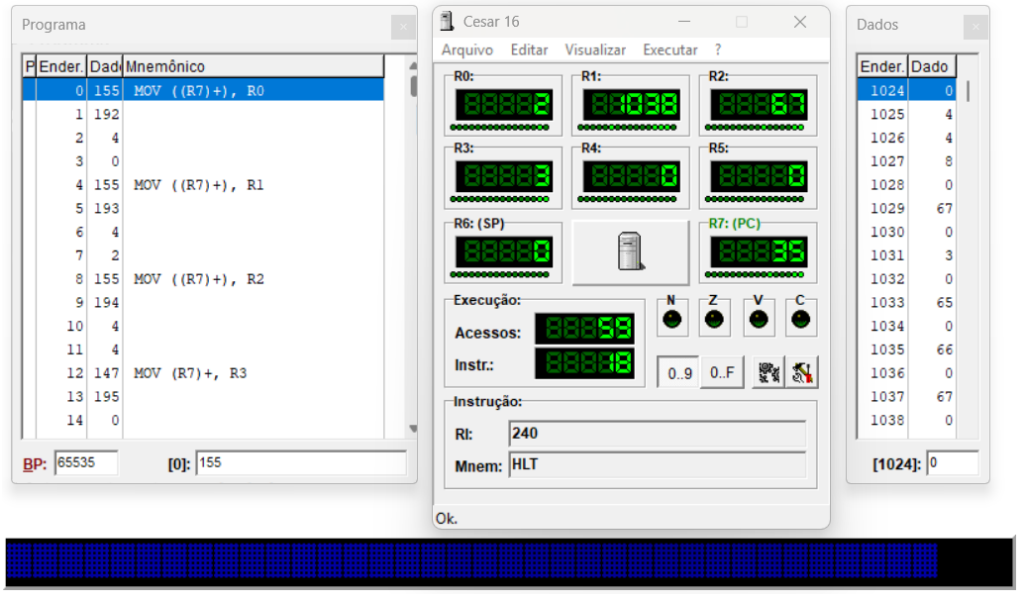
```
; Ex.1 - posicao de um valor (ASCII) em um vetor (1 = primeiro; 0 = ausente)
org 1024
    dw 4              ; n = tamanho do vetor
    dw 1032           ; endereco inicial do vetor
    dw 67             ; valor procurado: 'C'
    dw 0              ; resultado: posicao (1030)

org 1032
    dw 65             ; 'A'
    dw 66             ; 'B'
    dw 67             ; 'C'
    dw 68             ; 'D'

org 0
    MOV 1024, R0      ; R0 = n (elementos restantes)
    MOV 1026, R1      ; R1 = ponteiro do vetor
    MOV 1028, R2      ; R2 = valor procurado
    MOV #1, R3        ; R3 = posicao atual (comeca em 1)
    TST R0
    BEQ NAOACHOU      ; vetor vazio
LACO:
    CMP (R1)+, R2     ; compara o elemento e avanca o ponteiro
    BEQ ACHOU
    INC R3            ; proxima posicao
    SOB R0, LACO
NAOACHOU:
    CLR R3            ; nao encontrou: resultado 0
ACHOU:
    MOV R3, 1030      ; grava a posicao
    HLT
```

**Resultado obtido:** posição 1030 = 3, pois o valor 67 é o terceiro elemento do vetor. Para um valor ausente o resultado seria 0.

Obs. Como o valor 3 cabe em 1 byte, ele aparece no **1031** e o **1030** fica com 0




## Exercício 2: inicializar um vetor

> *Inicializar um vetor com valores definidos por constante ou por valor em memória.*

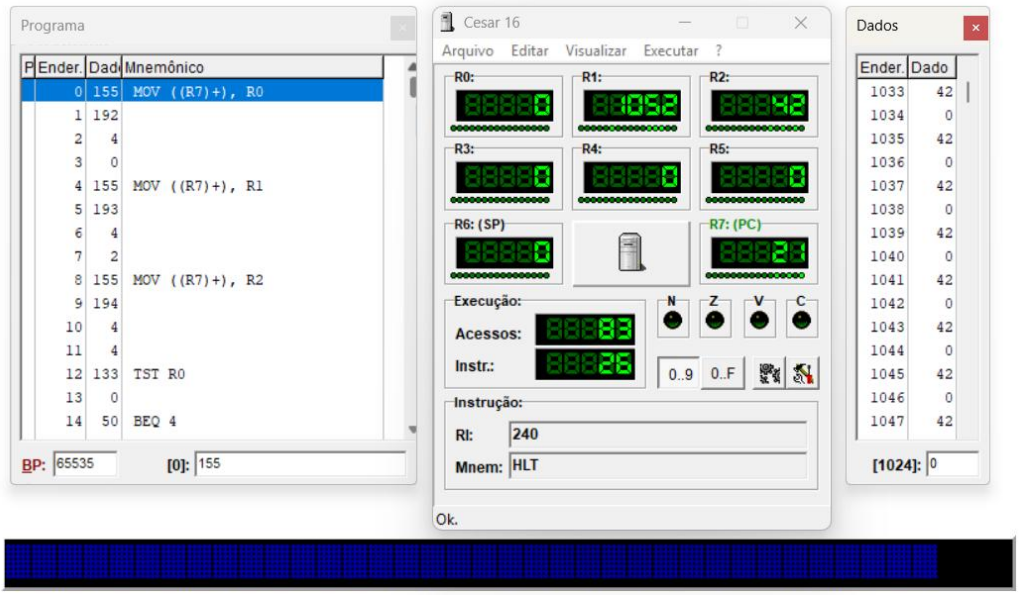
O programa lê o tamanho n, o endereço inicial e o valor a gravar de posições de memória, e grava o valor em cada posição do vetor com MOV R2, (R1)+, que escreve e avança o ponteiro em 2 bytes. O laço é controlado pelo SOB, e o teste de n = 0 evita que o contador seja decrementado a partir de zero. Para usar uma constante em vez do valor em memória, basta trocar MOV 1028, R2 por MOV #42, R2.

```
; Ex.2 - inicializar um vetor com um valor em memoria
org 1024
    dw 10              ; n = tamanho do vetor
    dw 1032            ; endereco inicial do vetor
    dw 42              ; valor a gravar ('*')

org 1032               ; vetor (10 palavras, inicialmente zeradas)
    dw 0
    dw 0
    dw 0
    dw 0
    dw 0
    dw 0
    dw 0
    dw 0
    dw 0
    dw 0

org 0
    MOV 1024, R0       ; R0 = n
    MOV 1026, R1       ; R1 = ponteiro do vetor
    MOV 1028, R2       ; R2 = valor
    TST R0
    BEQ FIM            ; n = 0: nada a fazer
LACO:
    MOV R2, (R1)+      ; grava e avanca o ponteiro
    SOB R0, LACO
FIM:
    HLT
```

**Resultado obtido:** as dez palavras a partir de 1032 (1032, 1034, ..., 1050) passam a valer 42, o código ASCII do caractere *.




## Exercício 3: manipulação de bits

> *Criar sub-rotinas para isolar um bit por máscara, zerar (clear) e ligar (set) um bit em uma posição indicada como parâmetro e contar o número de bits ligados.*

Os parâmetros são passados em registradores: R0 recebe o valor e R1, a posição do bit (de 0 a 15). A sub-rotina auxiliar MASCARA, chamada com JSR R4, constrói em R2 uma máscara com um único bit ligado, deslocando o valor 1 para a esquerda (ASL) R1 vezes. Com a máscara pronta, ISOLAR faz AND com o valor, mantendo apenas o bit desejado; ZERAR inverte a máscara (NOT) antes do AND, apagando só aquele bit; e LIGAR usa OR para forçar o bit em 1. A sub-rotina CONTAR trabalha em uma cópia do valor e executa 16 deslocamentos para a direita (ASR); a cada deslocamento, o bit que sai vai para o carry, e INC R1 conta os casos em que o carry é 1. As chamadas aninhadas usam registradores de ligação diferentes (R5 para as rotinas principais e R4 para a máscara), e a pilha é iniciada explicitamente em R6, com o valor 8192, para ficar longe dos dados e do visor.

```
; Ex.3 - manipulacao de bits (R0 = valor, R1 = posicao 0 a 15)
org 0
    MOV #8192, R6     ; inicia a pilha (SP), longe dos dados e do visor
    MOV VALOR, R0
    MOV #4, R1
    JSR R5, ISOLAR    ; isola o bit 4
    MOV R0, RES_ISOLAR
    MOV VALOR, R0
    MOV #5, R1
    JSR R5, ZERAR     ; zera o bit 5
    MOV R0, RES_ZERAR
    MOV VALOR, R0
    MOV #0, R1
    JSR R5, LIGAR     ; liga o bit 0
    MOV R0, RES_LIGAR
    MOV VALOR, R0
    JSR R5, CONTAR    ; conta os bits ligados
    MOV R1, RES_CONTAR
    HLT

MASCARA:              ; entrada R1 = pos; saida R2 = 1 deslocado pos vezes
    MOV #1, R2
    MOV R1, R3        ; R3 = contador de deslocamentos
LACO_M:
    BEQ FIM_M
```

```
    ASL R2
    DEC R3
    BR LACO_M
FIM_M:
    RTS R4

ISOLAR:             ; saida R0 = somente o bit da posicao R1
    JSR R4, MASCARA
    AND R2, R0
    RTS R5

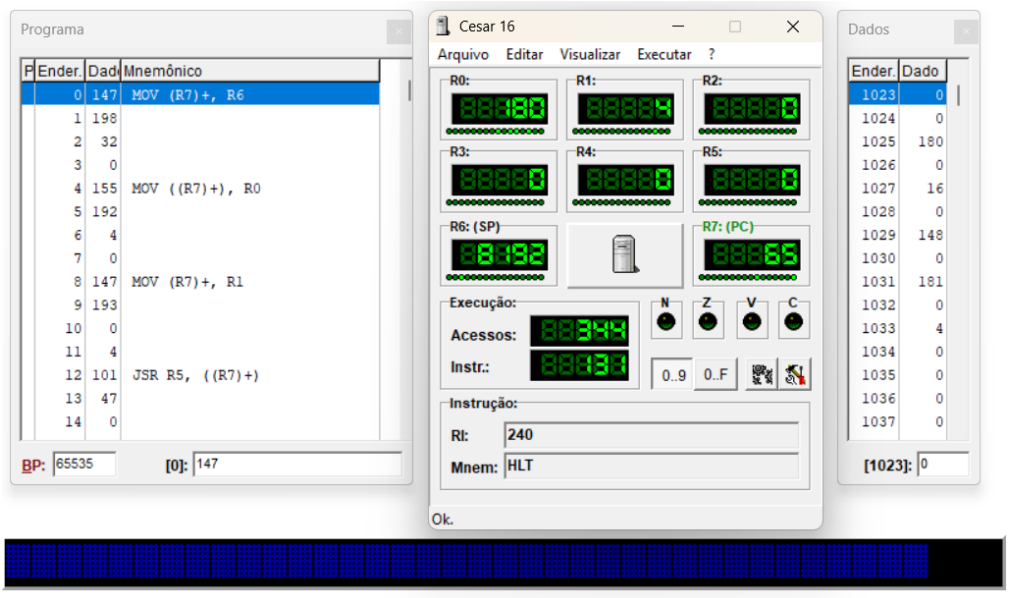
ZERAR:              ; saida R0 = valor com o bit da posicao R1 em 0
    JSR R4, MASCARA
    NOT R2          ; inverte a mascara
    AND R2, R0
    RTS R5

LIGAR:              ; saida R0 = valor com o bit da posicao R1 em 1
    JSR R4, MASCARA
    OR R2, R0
    RTS R5

CONTAR:             ; entrada R0 = valor; saida R1 = bits ligados
    CLR R1
    MOV R0, R2      ; trabalha em uma copia
    MOV #16, R3
LACO_C:
    ASR R2          ; bit menos significativo vai para o carry
    BCC PROX
    INC R1          ; carry = 1: bit ligado
PROX:
    SOB R3, LACO_C
    RTS R5

org 1024
VALOR:      dw 180  ; 10110100b
RES_ISOLAR: dw 0
RES_ZERAR:  dw 0
RES_LIGAR:  dw 0
RES_CONTAR: dw 0
```

**Resultado obtido:** para o valor 180 (10110100b): isolar o bit 4 resulta em 16, zerar o bit 5 resulta em 148, ligar o bit 0 resulta em 181 e a contagem de bits ligados é 4.



## 4.4 Trabalho avaliativo

> *Fazer um programa para mostrar nome e RA no visor do Cesar e apresentar um sinal de prompt (>) para esperar uma resposta do usuário do tipo S/N.*

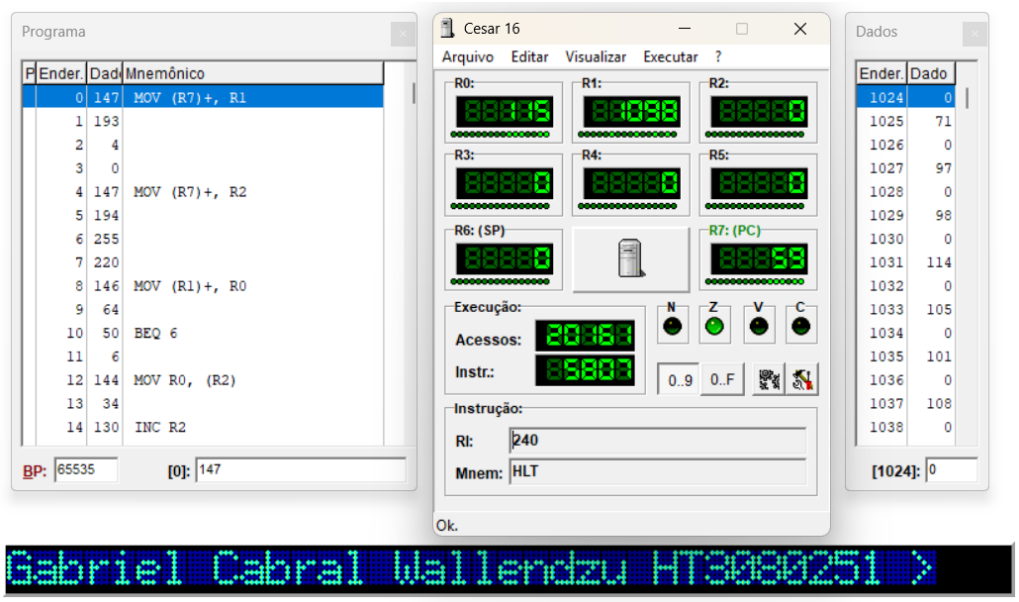
O texto “Gabriel Cabral Wallendzu HT3080251 >” tem exatamente 36 caracteres, que é a capacidade do visor, e por isso o nome, o RA e o prompt cabem juntos, com o sinal > na última posição. A string fica na área de dados, com um caractere por palavra (dw) e o valor 0 como terminador, porque a instrução MOV (R1)+, R0 lê palavras de 16 bits. O laço copia cada caractere para o visor, a partir de 65500; como o acesso à área de periféricos é sempre de 1 byte, apenas o byte menos significativo da palavra é gravado. Ao encontrar o terminador, o programa passa a esperar o teclado: libera o estado, aguarda uma tecla e lê seu código. As teclas S, s, N e n encerram o programa; qualquer outra é ignorada e o programa volta a esperar uma resposta.

```
; Trabalho avaliativo - nome, RA e prompt (S/N)
org 0
    MOV #TEXTO, R1    ; R1 = ponteiro da string
    MOV #65500, R2    ; R2 = ponteiro do visor
LACO_PRINT:
    MOV (R1)+, R0     ; le um caractere (palavra de 16 bits)
    BEQ LER_TECLA     ; 0 = fim da string
    MOV R0, (R2)      ; escreve 1 caractere no visor
    INC R2
    BR LACO_PRINT
LER_TECLA:
    CLR 65498         ; libera o teclado
ESPERA:
    TST 65498
    BEQ ESPERA        ; espera uma tecla
    MOV 65499, R0     ; le o codigo ASCII da tecla
    CMP #83, R0       ; 'S'
    BEQ FIM
    CMP #115, R0      ; 's'
    BEQ FIM
    CMP #78, R0       ; 'N'
    BEQ FIM
    CMP #110, R0      ; 'n'
    BEQ FIM
    BR LER_TECLA      ; outra tecla: ignora e espera de novo
FIM:
    HLT

org 1024
TEXTO:
    dw 71   ; G
    dw 97   ; a
    dw 98   ; b
    dw 114  ; r
    dw 105  ; i
    dw 101  ; e
    dw 108  ; l
    dw 32   ; (espaco)
    dw 67   ; C
    dw 97   ; a
    dw 98   ; b
    dw 114  ; r
    dw 97   ; a
    dw 108  ; l
    dw 32   ; (espaco)
    dw 87   ; W
    dw 97   ; a
    dw 108  ; l
    dw 108  ; l
    dw 101  ; e
```

```
    dw 110  ; n
    dw 100  ; d
    dw 122  ; z
    dw 117  ; u
    dw 32   ; (espaco)
    dw 72   ; H
    dw 84   ; T
    dw 51   ; 3
    dw 48   ; 0
    dw 56   ; 8
    dw 48   ; 0
    dw 50   ; 2
    dw 53   ; 5
    dw 49   ; 1
    dw 32   ; (espaco)
    dw 62   ; >
    dw 0                ; fim da string
```

**Resultado obtido:** o visor exibe “Gabriel Cabral Wallendzu HT3080251 >” e o programa fica aguardando; ao digitar S, s, N ou n, a execução termina em HLT, e outras teclas são ignoradas.




## 5 Testes realizados

Todos os programas foram montados no Daedalus e executados no simulador Cesar, no modo contínuo e passo a passo. Ao final de cada execução foram conferidos os registradores e as posições de memória, e os valores obtidos coincidiram com os esperados. A tabela resume as entradas utilizadas e as saídas verificadas.

| Programa | Entradas de teste | Saídas verificadas | Status |
|---|---|---|---|
| Soma de n posições (versões 0 a 4) | n = 5; vetor 10, 20, 30, 40, 50 | 1028 = 150; R0 = 150 | Aprovado |
| Soma de 32 bits | var1 = 1:60000; var2 = 2:10000 | 1032 = 4; 1034 = 4464 | Aprovado |
| Mover bloco | n = 4; origem 1030; destino 1034 | 1034 a 1040 = 11, 22, 33, 44 | Aprovado |

| Programa | Entradas de teste | Saídas verificadas | Status |
|---|---|---|---|
| Pesquisa em vetor | Vetor 12, 17, 23, 15, 8, 27, 26, 37 | 1028 = 37; 1030 = 8 | Aprovado |
| Teclado e visor | Teclas digitadas | Caracteres no visor, em sequência | Aprovado |
| Exercício 1 | Vetor A, B, C, D; procurado C | 1030 = 3 | Aprovado |
| Exercício 2 | n = 10; valor 42 | 1032 a 1050 = 42 | Aprovado |
| Exercício 3 | Valor 180 | 16; 148; 181; 4 | Aprovado |
| Trabalho avaliativo | Teclas S, s, N, n e outras | Nome, RA e > no visor; encerra com S/N | Aprovado |

# 6 Divisão das tarefas

O trabalho foi realizado de forma individual. Todas as etapas — escrita dos códigos, montagem, testes, organização do repositório no GitHub e redação deste relatório — foram executadas por Gabriel Cabral Wallendzu (HT3080251). As tarefas foram organizadas em um quadro kanban, mostrado em seu estado final.

| A fazer | Em andamento | Concluído |
|---|---|---|
| — | — | Soma de n posições: versões 0 a 4 |
| — | — | Exercícios resolvidos: 32 bits, bloco, vetor e teclado |
| — | — | Exercício 1: posição em vetor |
| — | — | Exercício 2: inicializar vetor |
| — | — | Exercício 3: manipulação de bits |
| — | — | Trabalho avaliativo: nome, RA e prompt |
| — | — | Repositório no GitHub e relatório |

# 7 Conclusão

O laboratório mostrou como o Cesar muda a forma de programar em relação ao Ramses. Com operações entre memória e registradores, modos de endereçamento como o pós-incrementado e instruções de controle como SOB, o mesmo algoritmo de soma ficou de 33 para 21 bytes e com o laço reduzido de cinco para duas instruções. Também ficou claro o cuidado exigido por uma arquitetura de 16 bits: os ponteiros avançam de 2 em 2, o carry precisa ser propagado entre partes de um número de 32 bits e o sentido da cópia de um bloco importa quando origem e destino se sobrepõem.

As sub-rotinas com JSR e RTS e os periféricos mapeados em memória completaram o estudo, e o trabalho avaliativo reuniu os dois: um programa que escreve no visor e depende do teclado. O uso do

Daedalus tornou o ciclo de escrita, montagem e simulação prático e facilitou a verificação dos resultados.

# 8 Repositório e referências

Os códigos-fonte de todos os programas deste relatório estão disponíveis na pasta Cesar do repositório do autor no GitHub:

https://github.com/GCWKGSWTGW/Computador-Cesar-Daedalus

Material de apoio da disciplina (Moodle do IFSP – Câmpus Hortolândia): roteiro do laboratório “Computador Cesar” e apostila do montador Daedalus. Referência da arquitetura: WEBER, Raul Fernando. Fundamentos de Arquitetura de Computadores. 2. ed. Porto Alegre: Sagra Luzzato, 1998.